

Labwork 5: Gaussian Blur Convolution

Ngo Quang Truong 2540106

3 Oct 2026

1 Introduction

This report presents an implementation of a 7×7 Gaussian blur filter on an RGB image, using three approaches: a sequential CPU implementation, a CUDA GPU implementation that reads directly from global memory, and a CUDA GPU implementation that uses shared memory.

2 Method

A Gaussian blur is a convolution between the image and a 2D kernel that follows a normal distribution:

$$G(x, y) = \frac{1}{2\pi\sigma^2} \exp \left[-\frac{(x - \mu_x)^2 + (y - \mu_y)^2}{2\sigma^2} \right] \quad (1)$$

The discrete, normalized 7×7 kernel used in this labwork is:

$$\frac{1}{1003} \begin{bmatrix} 0 & 0 & 1 & 2 & 1 & 0 & 0 \\ 0 & 3 & 13 & 22 & 13 & 3 & 0 \\ 1 & 13 & 59 & 97 & 59 & 13 & 1 \\ 2 & 22 & 97 & 159 & 97 & 22 & 2 \\ 1 & 13 & 59 & 97 & 59 & 13 & 1 \\ 0 & 3 & 13 & 22 & 13 & 3 & 0 \\ 0 & 0 & 1 & 2 & 1 & 0 & 0 \end{bmatrix}$$

For every output pixel, each color channel is replaced by the weighted sum of its 7×7 neighborhood with this kernel. Pixels outside the image border are handled with zero/edge padding (pad = 3 on every side, since $\lfloor 7/2 \rfloor = 3$).

Test image: a 1920×1080 RGB image.



Figure 1: Original RGB image

3 CPU Implementation

The CPU version pads the image, then loops over every pixel, every channel, and every kernel weight sequentially (three nested loops plus the 7×7 convolution window).

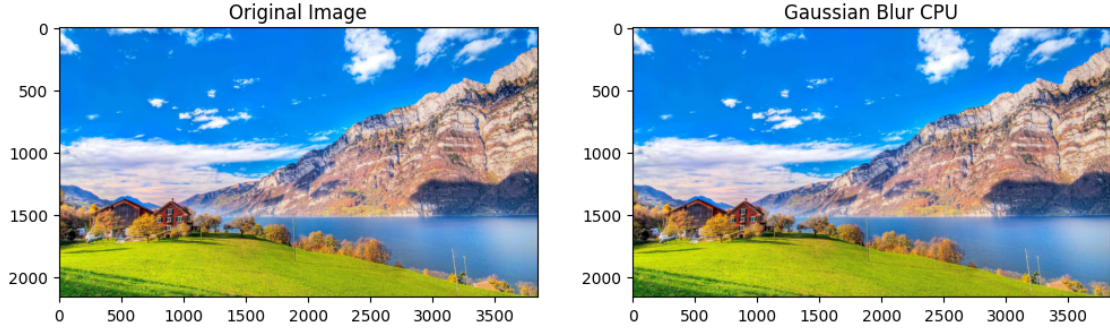


Figure 2: CPU Gaussian blur output

4 GPU Implementation without Shared Memory

Each CUDA thread is assigned one output pixel through `cuda.grid(2)`. The thread independently reads its own 7×7 neighborhood directly from global memory, multiplies by the kernel, and writes the result back.

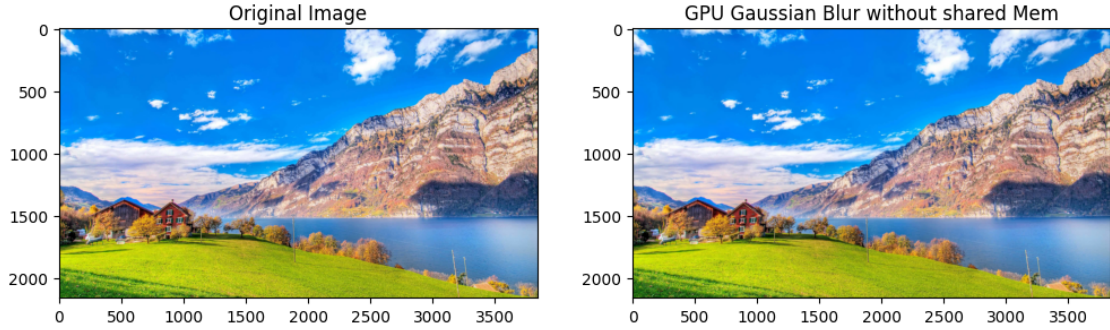


Figure 3: GPU Gaussian blur output (no shared memory)

5 GPU Implementation with Shared Memory

To reduce redundant global memory traffic, this version has every block cooperatively load its pixels into a shared memory tile of size $(16 + 2 \times 3) \times (16 + 2 \times 3) = 22 \times 22$ before any computation happens. A `cuda.syncthreads()` barrier ensures the tile is fully populated before any thread starts reading from it.

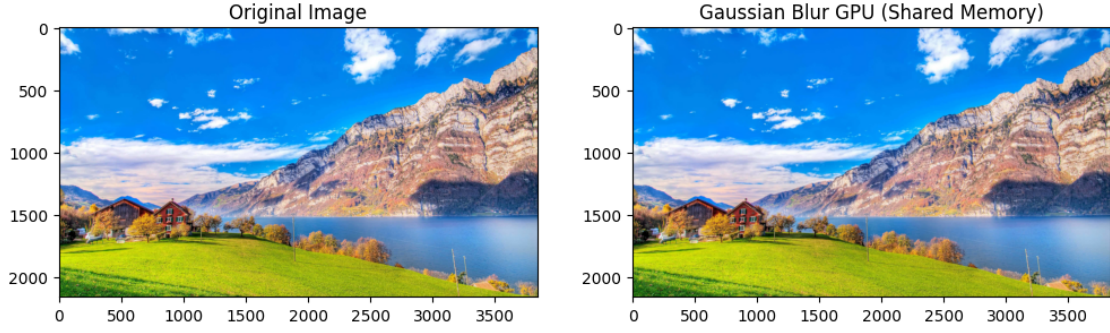


Figure 4: GPU Gaussian blur output (shared memory)

6 Results

Implementation	Time (s)	Speedup vs CPU
CPU (sequential)	174.747945	$1\times$
GPU, no shared memory	0.713428	$\approx 245\times$
GPU, shared memory	0.4778	$\approx 366\times$

Table 1: Execution time and speedup for the 7×7 Gaussian blur, 16×16 thread block.

Using shared memory further reduces the runtime from 0.713 s to 0.478 s, an additional $\approx 1.49\times$ speedup over the GPU version without shared memory.